

UNIT-3

PART 1:

Visualization Techniques: Spatial Data: One-Dimensional Data - Two-Dimensional Data – Three-Dimensional Data - Dynamic Data - Combining Techniques.

PART 2:

Geospatial Data: Visualizing Spatial Data- Visualization of Point Data -Visualization of Line Data - Visualization of Area Data - Other Issues in Geospatial Data Visualization

References:

1. Matthew Ward, Georges Grinstein and Daniel Keim, : Interactive Data Visualization Foundations, Techniques, Applications “,2010

3.1 Visualization Techniques for spatial data:

Spatial data visualization, which corresponds to the field of scientific visualization, assumes that the data has an implicit or explicit spatial or spatio temporal attribute.

This constraint facilitates both the creation and interpretation of the visualization, as there is an intuitive, and often straightforward, mapping of the data attributes to graphical attributes of the entities conveying the information in the visualization.

As our visual perception system is constantly receiving and interpreting imagery of the physical phenomena surrounding us, it is quite natural to process the same form of imagery on a computer screen.

In creating a visualization of spatial data, we must decide what spatial attributes of the data will map to the spatial attributes (locations) on the screen.

This can involve many forms of transformation, including scaling, rotation, translation, shearing, and projection.

Once the spatial attributes are accommodated, other attributes and values associated with the data must then be mapped to other components of the visualization, whether it is an attribute such as color or texture or the size or shape of a graphical entity.

In creating a visualization of spatial data, we must decide what spatial attributes of the data will map to the spatial attributes (locations) on the screen. This can involve many forms of transformation, including scaling, rotation, translation, shearing, and projection. Once the spatial attributes are accommodated, other attributes and values associated with the data must then be mapped to other components of the visualization, whether it is an attribute such as color or texture or the size or shape of a graphical entity.

One-Dimensional Data:

One-dimensional spatial data is often the result of accumulating samples or readings of some phenomenon while moving along a path in space. For example, a drill-hole sample

will contain mineral content and ore grade information based on the distance from the top of the drill-hole. This sort of sampling is often referred to as a probe when exploring structures of higher dimensions.

Given a one-dimensional sequence of univariate data (only one value per data item), we can map the spatial data to one of the screen dimensions and the data value itself to either the other screen dimension (to form a line graph, see Figure) or to the color of a mark or region along the spatial axis (to form a color bar). The data needs to be scaled to fit within the range of the display attribute (either number of pixels or number of colors). Parts of the display

space or color range might be reserved for other aspects of the visualization, such as the axes, labels, and key, so the most general structure for an algorithm to generate such a visualization will use parameters for the bounds of both the data and display spaces.

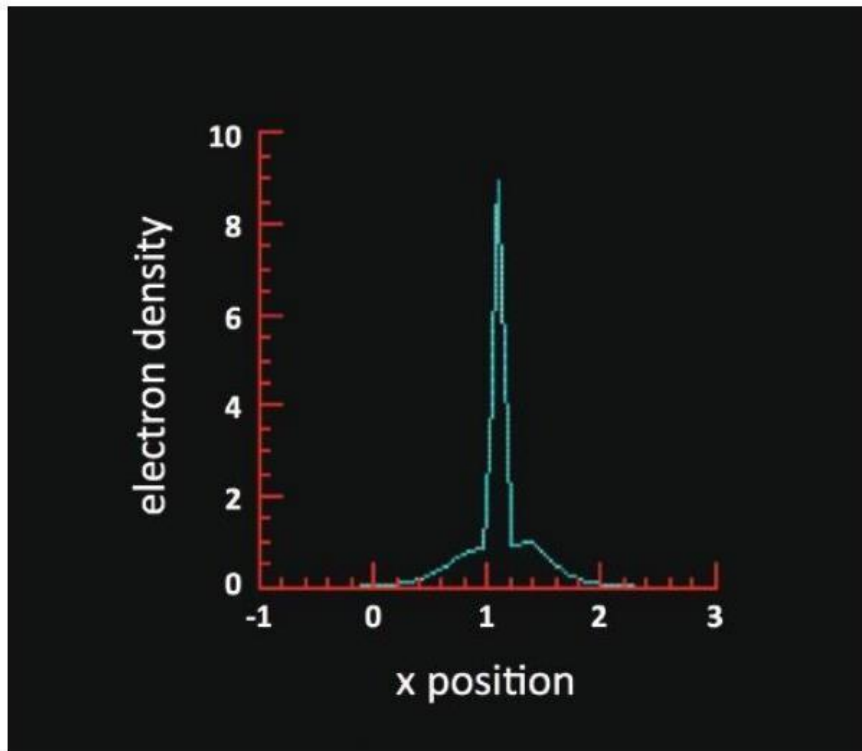


Fig : A line graph of a 1D sequence of data values

Assume that (datamin, datamax) are computed as the minimum and maximum values for the data, and datacount indicates the number of data points to be displayed (this could be all of the data, or just a selected subset). Also assume the section of the display that will hold the visualization is defined by the rectangle (xmin, ymin, xmax, ymax). To generate a line graph from an array called “data,” we could use the following code (assuming the drawing command draw-Line exists).

Draw-Line-Graph(data, dataCount, xMin, xMax, yMin, yMax)

```
1 dataMin ← computeMin(data, dataCount)
2 dataMax ← computeMax(data, dataCount)
3 xFrom ← xMin
4 yFrom ← worldToScreenY(data[0], dataMin, dataMax, yMin,
5 yMax)
6 for i ← 1 to dataCount
7 do xTo ← worldToScreenX(i, dataCount, xMin, xMax)
8 yTo ← worldToScreenY(data[i], dataMin, dataMax,
9 yMin, yMax)
10 drawLine(xFrom, yFrom, xTo, yTo)
11 xFrom ← xTo
12 yFrom ← yTo
   worldToScreenX(index, dataCount, xMin, xMax)
return (xMin + index * (xMax - xMin)/dataCount)
worldToScreenY(value, dataMin, dataMax, yMin, yMax)
return (yMin + (value - dataMin) * (yMax - yMin)/(dataMax - dataMin))
```

Note that each transformation is a combination of offsets and scaling that we use to convert values in one coordinate system into another. This operation is used heavily in computer graphics, and we will use variants on it numerous times in this book. It is derived based on the observation that a point within a given range can be transformed to a point within a different range by computing the relative position of the point within the range, e.g., what percentage of the distance from one end of the range to the other is the selected point. Mathematically, this is given as

$$(A_i - A_{\min}) / (A_{\max} - A_{\min}) = (B_i - B_{\min}) / (B_{\max} - B_{\min})$$

where A_i is a location in the range $[A_{\min} \rightarrow A_{\max}]$ and B_i is the corresponding location in the range $[B_{\min} \rightarrow B_{\max}]$.

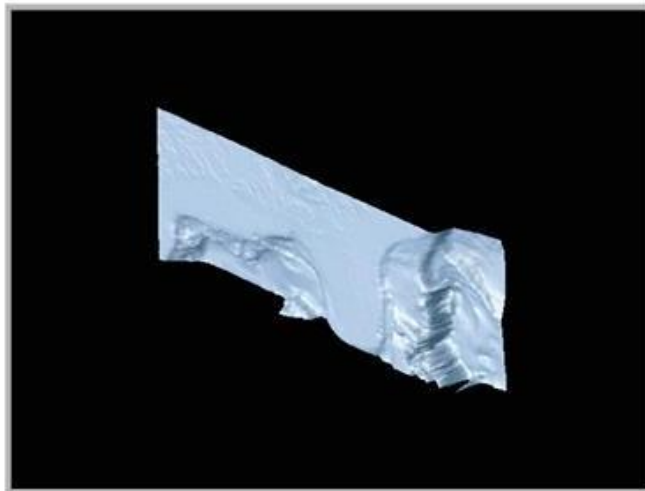
Two-Dimensional Data:

Data with two spatial dimensions get visualized predominantly by mapping the spatial attributes of the data to the spatial attributes of the screen. The result can be one of the following visualizations:

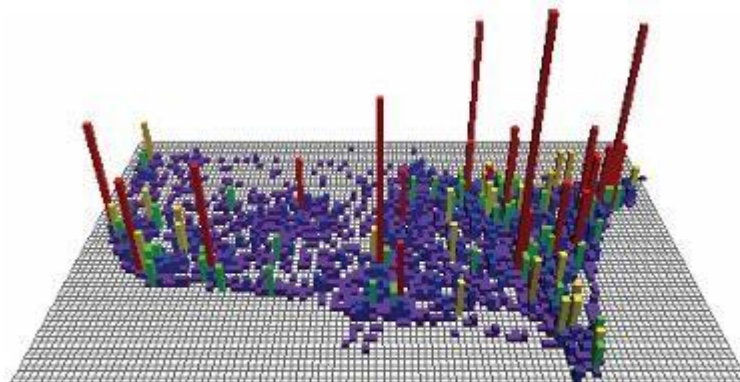
1. **An image** results if a single data value at each location is mapped to color and all intermediate pixels are colored via interpolation .
2. A **rubber sheet** results if the data, whether regularly or irregularly spaced, is mapped to the height of a point in three dimensions, with the points triangulated so that a surface can be formed.
3. A **cityscape** is formed by drawing three-dimensional objects (generally boxes) at locations on a plane, where the data can control the attributes of the graphical objects (i.e., height and color).
4. A **scatterplot** results if, at each location on the plot, the data value(s) control the color, shape, or size of a marker. Note that unlike for images, no interpolation is performed.
5. A **map** results if the data contains linear and area features, as well as point objects. A linear feature, such as a road or stream, is represented as a sequence of connected coordinates, which are plotted as a series of line segments. Area features, such as a lake or political boundary, are generally represented as a closed contour, a set of coordinates where the first and last points are the same.
6. A **contour or isovalue map** conveys boundary information extracted from an image depicting a continuous phenomenon, such as elevation or temperature. The term isovalue means “single value,” and thus a contour on such a map indicates the boundary between points above this value and points below the value. It can be formed by considering two-by-two arrays of adjacent data values as the corners of a rectangle or square, and generating edges across this rectangle when one or more values are on the opposite side of the isovalue from one or more of the others.



An image from a tomographic data set.



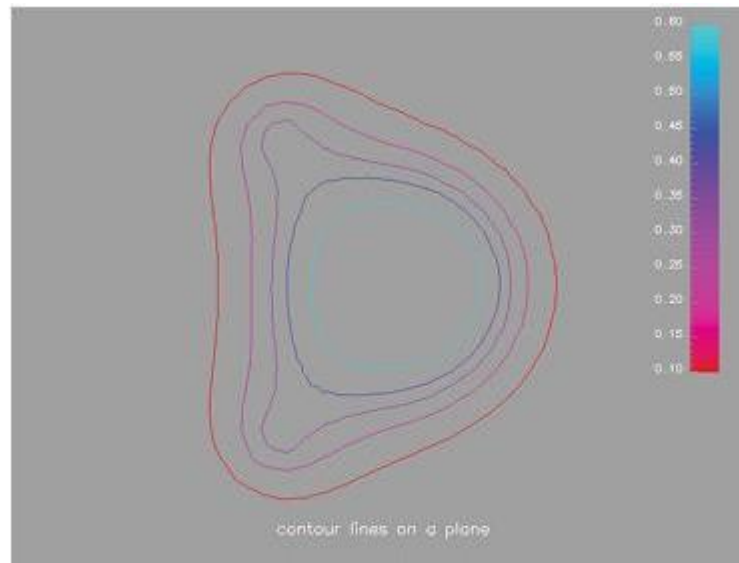
A Rubber Sheet visualization of elevation data for the southeast U.S. (Image generated using OpenDX [308].)



A cityscape showing the density of air traffic over the United States at a particular time period.



A map of the San Francisco area, generated by Google Maps. (Image © 2008 Google Map; data © 2008 NAVTEQ™.)



Contour lines from an image slice of a hydrogen molecule. (Image generated using OpenDX [308].)

Probing Two-Dimensional Data

Besides the visualization techniques that display the entire data set, we can also choose to visualize one-dimensional subsets, projections, or summarizations of the data. After reducing the dimensionality in some manner, we are then free to employ any of the techniques described in the previous section. Let us examine some of the projection options in detail.

Frequency histograms. We can summarize data sets of arbitrary dimensionality by computing the frequency at which values or subranges of values occur in the data, and then display this information as a bar graph. Important considerations when using subranges are how to decide the number of sub ranges to use, and where the breaks between subranges should be. Features of importance in the data might be lost if this is done incorrectly, and it is best performed by someone familiar with the data characteristics. Simply dividing the range into a fixed number of evenly sized subranges, while simple, is rarely an effective strategy.

Row and column aggregations. Visually depicting summarizations of the rows and/or columns of an image is a useful mechanism for localizing the boundaries of features of interest and regions of low or high variability. Many descriptors can be used for this purpose, including the sum, average, median, standard deviation, maximum, or minimum values. The resulting one dimensional visualization can be viewed separately or, as often the case, placed alongside the two-dimensional visualization as supplemental information. Color bars, line plots, and bar graphs have all been used with success in this type of visualization.

Linear probes. A one-dimensional probe of a two-dimensional data set can be likened to a drill hole for mineral exploration. A line is passed through the data, and the values encountered are visualized using one of the one dimensional techniques previously described. To accomplish this, we use two mathematical tools already presented in this chapter: parametric equations and bilinear interpolation. We start by creating a parametric equation of our probe using input from the user (either a pair of points or a point and a direction). If we assume that this line was specified as two sets

of coordinates, P_1 and P_2 , the parametric equation of the line segment joining the points is simply $P(t) = P_1 + t(P_2 - P_1)$, where $0 \leq t \leq 1.0$. We can then get the coordinate of an arbitrary point along this line segment by choosing a value of t within this range. Generally, these are evenly spaced, and the number of samples computed is proportional to the length of the line segment (so that probes that barely clip a corner of the data grid do not get represented by the same number of samples as a probe along the diagonal of the grid). Given the coordinates of the sampled points, we can now use the interpolation procedure to compute values, which can then be visualized as a one-dimensional data set

Three-Dimensional data: As with two-dimensional data, three-dimensional spatial data may be either discrete samples of a continuous phenomenon or a structure best described via vertices, edges, and polygons. In reality, many visualizations of science and engineering data contain a combination of these data representations, such as air flow around a wing or stress attributes of a mechanical part.

1. Visualizing Explicit Surfaces:

An explicit surface is one that has been defined in one of two ways:

1. a list of three-dimensional vertices, a list of edges, i.e., connections between the vertices (specified as a pair of indices into the vertex list), and a list of planar polygon patches (usually specified as a fixed or variable length list of indices into the edge list);
2. a set of parametric equations for defining the x-, y-, and z-coordinates of points on the surface, along with an interconnection strategy (e.g., a triangular or rectilinear grid) for computing the edges and patches. The step size of the parameters can be used to control the smoothness of the curved surface.

2. Visualizing Volume Data:

As pixels are to two-dimensional visualization, voxels, or volume elements, are to three-dimensional visualization. Volume data is generally a sampling of a continuous phenomenon, and can be either acquired via sensors (e.g. tomographic data sets) or generated via simulations (e.g., computational fluid dynamics). In each case, we have one or more data dimensions with regular or irregular positions, and the goal is to convey to the viewer the structure, patterns, and anomalies within the data.

Most approaches to visualizing volume data fall into one of the following categories :

Slicing techniques. Using a cut plane, either aligned with an axis or arbitrarily oriented, probe the data to extract a two-dimensional slice of the data, and then use one of the two-dimensional spatial data visualization methods.

Isosurface techniques. Given a user-specified value, generate a surface description and visualize it using one of the explicit surface visualization

techniques.

Direct volume rendering. Either cast rays into the volume and compute a pixel value based on the data encountered by the ray, or project each voxel onto the projection plane using some method of accumulating effects on pixels.

Direct Volume Visualization Techniques. Direct volume rendering means that no three-dimensional polygons are created for use with traditional graphics rendering techniques. Rather, pixels in the resulting image are computed on an individual basis, either by casting rays through the pixel through the volume, or by projecting voxels onto the plane of projection.

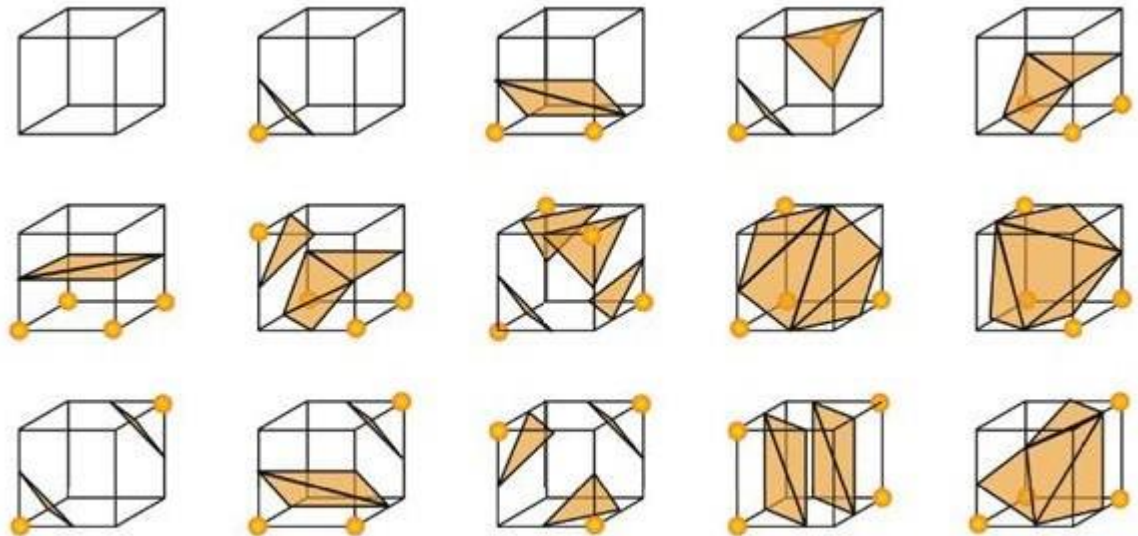


Figure 5.7. The 15 voxel configurations for Marching Cubes. (Image courtesy Wikimedia Commons.)

The basic process of rendering a volumetric data set starts by transforming the positions of the voxels into the viewing coordinate system, following the same procedure used in the traditional three-dimensional graphics pipeline. This may or may not include perspective distortion. The viewer must specify a view reference point (the origin of the plane of projection), a view direction (the normal to the plane of projection), the height and width of the image to be projected on the plane of projection, and, for a perspective projection, the distance from the camera to the plane of projection. The reader is referred to textbooks on computer graphics for details of this process.

Once the voxels have been positioned, we have the option of either:

forward mapping—project each voxel onto the plane of projection and determine which pixels will be affected and in what way;

inverse mapping, also called ray casting—send a ray from each pixel in the plane of projection through the volume, sampling values along the ray, and determining the resulting value for each pixel.

Implicit Surfaces:

Typical method of modeling surfaces in computer graphics

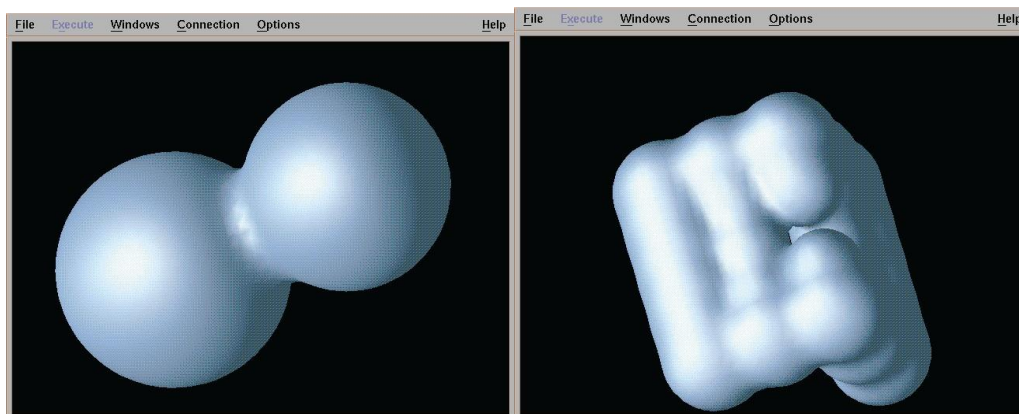
The typical method of modeling surfaces in computer graphics is using parametric equations to define points on the surface, which can then be connected to form polygonal meshes. This representation is very useful for performing transformations and computing surface normals.

Implicit representations

An alternative method is the use of implicit representations [41], where the surface is defined as the zero contour of a function of two or three variables. Implicit representations have strengths in operations such as blending and metamorphosis, and have been becoming increasingly popular. They can also be used in data visualization to convey sparse data sets and the field of influence for the data points. They also enable combining clouds of data into higher-level surfaces and solids.

Metaballs / Blobby objects

Metaballs, also known as blobby objects [475], are a type of implicit modeling technique. We can think of a metaball as a particle surrounded by a density field, where the density attributed to the particle (its influence) decreases with distance from the particle location. A surface is implied by taking an isosurface through this density field—the higher the isosurface value, the nearer it will be to the particle. The powerful aspect of metaballs is the way they can be combined. By simply summing the influences of each metaball on a given point, we can get very smooth blendings of the spherical influence fields.



5.10 Examples of implicit surfaces. The figure on the left shows two intersecting spheres, while the figure on the right contains a bent grid of 25 spheres with different ranges of influence, where the center sphere is missing.

Influence equation

The key to using metaballs is the definition of the equation for specifying the influence on an arbitrary point from an arbitrary particle. Blinn [40] used exponentially decaying fields for each particle, with a Gaussian bump with height b and standard deviation a . If r is the distance from the particle to a location in the field, the influence of the particle is $[b \text{ to the power } (-ar)]$. For efficiency purposes, this was changed to the squared distance to avoid computing the square root. The resulting density of an arbitrary location in the field is simply the summation of the contributions from all the particles.

Dynamic Data

Flow Visualization

Flow visualization is the study of methods to display the dynamic behavior in liquids and gases. The field dates back at least to the mid-1400s, where Leonardo Da Vinci sketched images of fine particles of sand and wood shavings that had been dropped into flowing liquids.

Since then, laboratory flow visualization has become more and more exact, with careful control of the particulate size and distribution. Advances in photography have also helped extend our understanding of how fluids flow under various circumstances.

Computational Fluid Dynamics (CFD)

More recently, computational fluid dynamics (CFD) has extended the abilities of scientists to study flow by creating simulations of dynamic behavior of fluids under a wide range of conditions. The result of this analysis is usually a 2D or 3D grid of velocity vectors, which may be uniformly or nonuniformly spaced. The goal is then to analyze this vector field to identify features such as turbulence, vortices, saddle points, and other forms of structure.

Variations in Flow Field Data

There are several variations on the structure of the field data that is generated by these experiments. A static field is one in which there is only a single, unchanging velocity field. Time-varying fields may either have fixed positions with changing vector values or both changing positions and changing vectors (for example, when modeling rotating turbine blades or pitching airfoils). These latter types are referred to as unsteady.

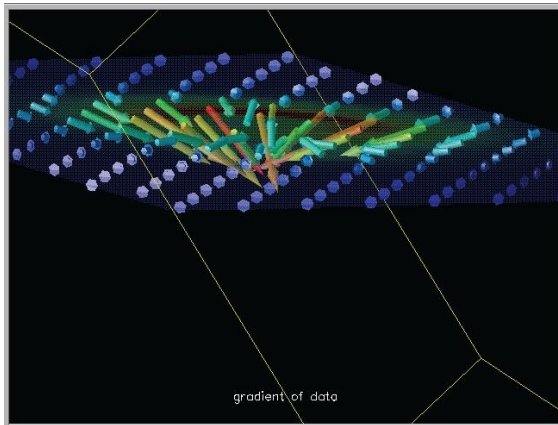
Definitions

We provide some definitions that we will need in our discussion of the mathematics of particle advection.

- pathline—The trajectory of a particle released into a flow field; it assumes multiple time instances. In experimental visualization, this can be achieved by long-term film exposure.
- streakline—Simultaneous positions of a set of particles continuously released from one or more locations.
- timeline—Position at an instant of time of a batch of particles that have been released simultaneously.
- streamline—A line through the velocity field that is tangent to the velocity field at every point.
- steady flow—A flow field that does not change with time. For steady flow, streaklines, pathlines, and streamlines coincide.
- particle advection—Computing the motion of particles through a flow field.
- vorticity—The curl of the velocity field, giving the magnitude and direction of angular velocity for each particle in the velocity field.

Visualization Techniques

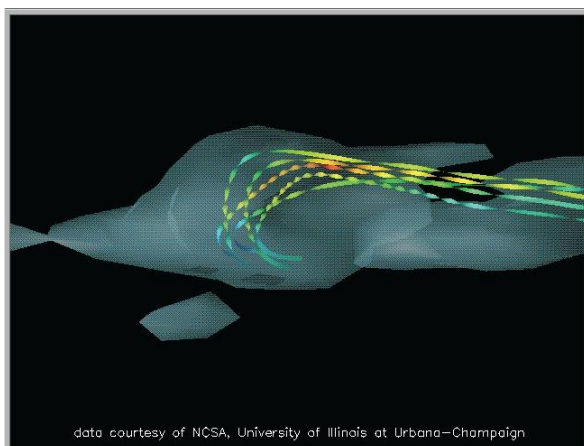
Many methods for visualizing flows have been developed over the years [21]. The simplest form of flow visualization is to display the velocity field data itself, either as displacement vectors using arrow glyphs (see Figure 5.11) or as magnitude scalar values using image, surface, or volume visualization techniques (mapping values to color, size, or position). The number and placement of the displayed components is a crucial factor in conveying the



5.11 A storm cloud visualization containing glyphs showing wind direction and strength

important features of the data; showing too many flow field components can lead to significant occlusion, while using too few elements raises the potential for missing features. While the simplest solution is to allow the user to interactively change the density of elements (often specified by a plane or vector in space, along with a sampling rate), recent research has focused on automated placement of the displayed flow field components by analyzing the data and identifying regions where potentially interesting flow is occurring.

The next most common technique is the generation of streamlines based on a static velocity field. The user selects seed locations (often along a line or at two-dimensional grid locations), and computes a path for each seed point through the field, maintaining a continuous tangent to the flow field. Besides using lines to indicate the streams, we can use planar or solid objects, such as ribbons and tubes. Other attributes of the field, such as magnitude or vorticity, can now be mapped to other attributes of the streamribbons or stream-tubes, such as color, size, or twist (see Figure 5.12).



5.12 Flow data visualized using ribbons, with vorticity mapped to twist.

Streaklines are often represented as a continuous stream of particles emanating from a discrete set of points and flowing through the field (see Figure 5.13). Individual points in a given trace (all particles coming from a particular location belong to the same trace) may be identified by color-coding to help distinguish related points that get separated when entering areas of high velocity.



5.13 An example of streaklines: Sparks from a Campfire

Line Integral Convolution (LIC)

Line Integral Convolution. An interesting approach to vector field visualization was developed by Cabral and Leedom in 1993 [54]. The method, called line integral convolution, or LIC, uses a random field and a vector field with the same height and width to generate a dense display of flow information (see Figure 5.14). Basically, every pixel in the resulting image is a weighted average of a sequence of adjacent pixels in the random field along a linear path centered on the given pixel and following the streamline going through the pixel.



5.14 The texture field on the left is combined with the vector field in the middle to create the line integral convolution on the right.

More formally, we can start at each location (i, j) in the vector field and create two chains of pixels emanating from this location in opposite directions tangent to the vector orientation.

Combining Techniques:

Combined Flow Visualization Techniques

Many effective visualizations are actually combinations of two or more of the techniques. Each technique has its strengths and weaknesses in terms of the types of information it can or cannot effectively visualize, so a combined visualization, as long as occlusion is minimized, can generate results that are the sum of the strengths. At the same time, more and more problems require the simultaneous analysis of multiple data sets to arrive at an informed result.

For example, weather forecasting involves the combination of air and surface temperature, wind speed, relative humidity, and a number of other factors to develop an accurate prediction. In this section we will examine a number of visualizations formed by combining some of the methods previously discussed, highlighting factors important to their successful design and interpretation.

A. Slice Plus Isosurface:

The isosurface is capable of conveying surface structure, which is difficult to obtain from volume slicing, even with animation of the slice position. However, the isosurface only provides information on a single value within the entire volume, with no indication of the distribution of other values or the gradient (rate of change) of the selected value at different locations.

The slice provides very detailed two-dimensional information, especially with an appropriate choice of color assignments. It can convey to the user the regions of relative uniformity, as well as those exhibiting significant change. Another advantage is that the image slice can convey nested regions of a particular value range, while the isosurface, in general, will only display the outer-most surface.

Several considerations and capabilities are important when designing and developing this visualization:

- it is critical to support rapid and intuitive modification of the isosurface value;
- the position and orientation (along the three axes) of the slice should be easily controlled by the user, with the option for animating its position as it steps through the volume;
- camera position and orientation control are essential to enable viewing of the surface and slice from different orientations;
- color assignment is crucial, as features of interest in the slice may only be revealed with a carefully chosen color map. Similarly, we might want to choose the color of the isosurface so that it does not appear in the color map used for the slice, to reduce the potential for misinterpretation;
- the user should be able to easily hide either of the two visualization components to make it easier to study the remaining data display. Another option is to allow the user to control the opacity of the components, perhaps using a slider that ranges from hiding the slice entirely to hiding the isosurface.

B. Isosurface Plus Glyphs:

Isosurfaces are useful for conveying details of three-dimensional surfaces, but, in general, do not incorporate other aspects of the data. Glyphs, such as the popular arrow glyph, can be used to display the magnitude and direction of change within a data set, either as a gradient in static data or a flow in dynamic data. The glyphs may be positioned in close proximity to the isosurface, since these are known to be positions of interest, or the positions may be controlled separately

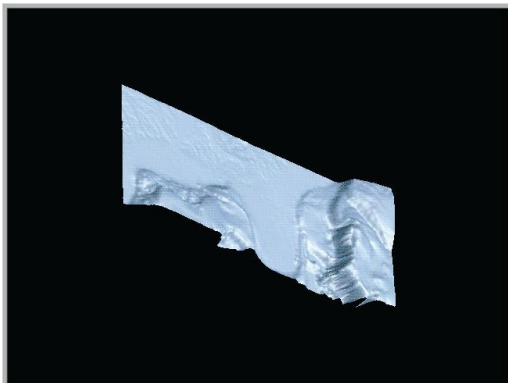
Many of the design considerations from the previous example pertain to this visualization as well. Interactive control of the parameters of the visualization—the isosurface value, glyph base position,

and viewing position and angle—contribute to making this an effective visualization. Other controls are necessary for improving the information content of the glyphs, including:

- varying the density of the glyphs;
 - scaling the glyph sizes;
 - controlling the color of glyphs to convey additional information or redundantly map to the wind magnitude
- computing base positions for the glyphs based on regions of interest in the vector or water density fields.

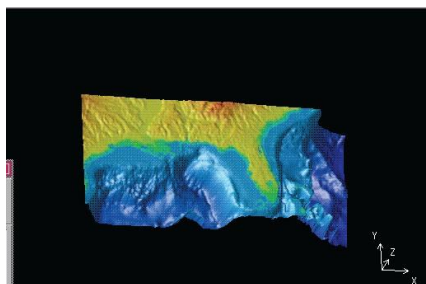
C. Rubber Sheet plus Contour Lines and Color:

We have mentioned several times the benefits of redundant mappings of the same data to different attributes of the visualization. In this example, we start with a rubber sheet to convey a two-dimensional field of values as a height field. This can reveal peaks and valleys found in the data, and by creating a virtual landscape, we can build on the users' intuition about interpreting the data in a topographic manner (see Figure 5.17).

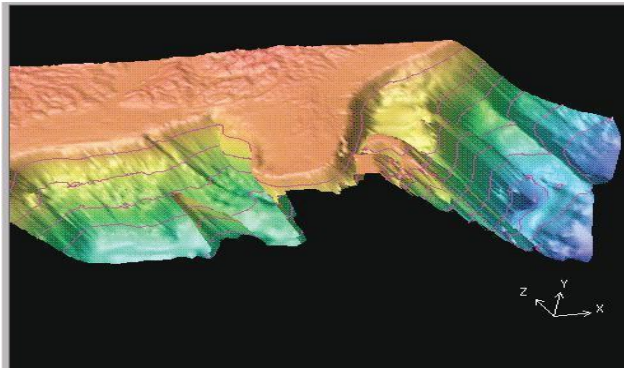


5.17 A rubber sheet display of the height above and below sea level for the southeast United States

We can augment this visualization by mapping color to the elevation, thus making it easier to identify widely separated regions of similar height (see Figure 5.18).



5.18 The same data as in Figure 5.17, with color mapped to elevation



5.19 A zoomed-in region of the same data as the previous two figures, with additional contour lines.

Finally, we can superimpose contour lines at certain levels, making the gradient information much more apparent (see Figure 5.19). Note that each addition to the visualization has improved our understanding of the data, yet there is some value to viewing the data in multiple ways. This reinforces our design consideration regarding selective hiding and showing of visualization components to take advantage of their individual strengths while enabling users to incrementally enhance their mental model of the data contents.

Unit 3 PART 2:

2. Geospatial data

Geospatial data is different from other kinds of data in that spatial data describes objects or phenomena with a specific location in the real world. Geospatial data arises in many applications, including credit card payments, telephone calls, environmental records, and census demographics. In this chapter, we provide an overview of the special characteristics and methods that are needed for the visualization of geospatial data, sometimes called geo-visualization.

2.1 Visualizing Spatial Data

Large spatial data sets can be seen as a result of accumulating samples or readings of phenomena in the real world, while moving along two dimensions in space. Often, spatial data sets are discrete samples of a continuous phenomenon. Nowadays, there exists a large number of applications where it is important to analyze relationships that involve geographic location.

Examples include global climate modeling (e.g., measuring temperature, rainfall, and wind speed), environmental records (e.g., measuring CO₂ and other pollution levels), economic and social measures and indicators (e.g., unemployment rate, education level), customer analysis, telephone calls, credit card payments, and crime data. Because of its special characteristics, the basic visualization strategy for spatial data is straightforward; we map the spatial attributes directly to the two physical screen dimensions, resulting in map visualizations.

Maps are the world reduced to points, lines, and areas. The visualization parameters, including size, shape, value, texture, color, orientation, and shape, show additional information about the objects under consideration. According to the U.S. Geological Survey (USGS), map visualizations are defined as a set of points, lines, and areas, all defined both by position reference to a coordinate system (spatial attributes) and by their nonspatial attributes. MacEachren defines geographic visualization as the use of visual representations to make spatial contexts and problems visible, so as to engage the most powerful human processing abilities, those associated with vision.

From the definitions, it becomes clear that we may distinguish spatial phenomena according to their spatial dimension or extent:

point phenomena—have no spatial extent; they can be termed zero dimensional and can be specified by a longitude and latitude coordinate pair, along with a set of descriptors or attributes. Examples are buildings, oil wells, aggregated measures, and cities.

line phenomena—have length, but essentially no width; they can be termed one-dimensional and can be specified by an unclosed series of longitude and latitude coordinate pairs for each phenomenon. Examples are large telecommunication networks, roads, and boundaries between countries. Attributes associated with line phenomena might include capacities, traffic levels, and names.

Area phenomena—have both length and width; they can be termed two-dimensional and can be specified by a series of longitude and latitude coordinate pairs that completely enclose a region, along with a set of attributes for each phenomenon. Examples are lakes, parks, and political units such as states or counties.

surface phenomena—have length, width, and height; they are termed two-and-half-dimensional and can be specified by a series of longitude, latitude, and height coordinate vectors with a set of attributes for each (longitude, latitude) pair.

Maps can be subdivided into map types based on properties of the data (qualitative versus quantitative; discrete versus continuous) and the properties of the graphical variables (points, lines, surface, volumes). Examples of the resulting maps are

- symbol maps (nominal point data);
- dot maps (ordinal point data);
- land use maps (nominal area data);
- choropleth maps (ordinal area data);
- line diagrams (nominal or ordinal line data);
- isoline maps (ordinal surface data);
- surface maps (ordinal volume data).

Note that the same data may be visualized by different map types. By aggregating point data within areas, a choropleth map may be generated out of a dot map, or a land use map out of a symbol map. We may also generate a density surface from a dot map and display it as an isoline map or a surface map. If we aggregate the point data within areas and map the number of points within the areas to their size, we obtain cartogram visualizations.

In exploratory geo-visualization, interaction with maps is crucial. In contrast to traditional cartography, the classification and mapping of the data can be interactively adapted by the user, and interactive querying as well as manipulation of the display are possible. A number of techniques and systems have been developed that make extensive use of such interaction capabilities. They allow, for example, a linking of multiple maps or a combination of maps with standard statistical visualizations, such as bar charts and line charts, or even with complex multidimensional visualization techniques such as parallel coordinates or pixel techniques. In addition, they usually provide an advanced browsing or querying interface.

Map projections are methods for transforming positions on the Earth's spherical surface into positions on a flat map. Each location on the Earth is given by longitude (λ) ranging from -180° to 180° and latitude (ϕ) ranging from -90° to 90° .

Because the globe is curved and maps are flat, projections always introduce some distortion. They are designed to preserve one property while distorting others. Major projection types include:

- **Conformal projections** – Preserve local angles and shapes but distort area.
- **Equal-area (equivalent) projections** – Preserve area size everywhere but distort shapes or angles.
- **Equidistant projections** – Preserve true distances from one or more specific points or lines on the map.
- **Gnomonic projections** – Show all great circles (shortest paths on Earth) as straight lines, useful for navigation.
- **Azimuthal projections** – Preserve direction from a central point; distances from that point are consistent in all directions.
- **Retroazimuthal projections** – Preserve the direction from any point on the map to one fixed location.

Map projections can also be classified based on the geometric surface used for projection, such as a cylinder, cone, or plane.

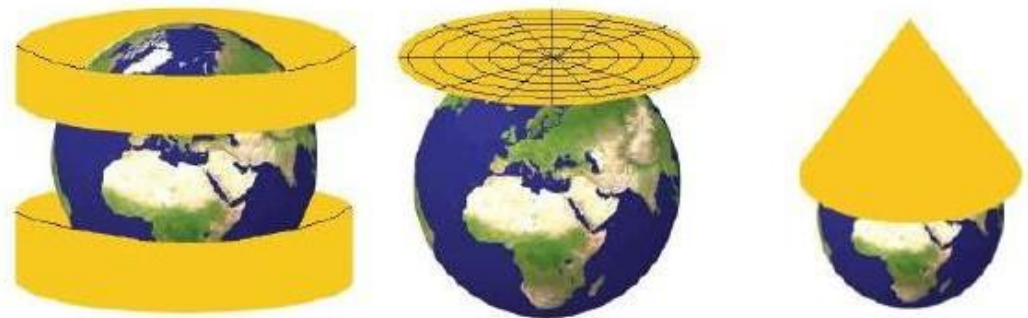


Figure 6.1. Cylinder, plane, and cone projections.

Map projections can be classified by the geometric surface used to unwrap the globe:

1..Cylindrical Projections

- The Earth's surface is projected onto a cylinder wrapped around the globe.
- The entire world can be displayed.
- Typically **conformal** → preserve local angles and shapes (but distort area).
- Longitude and latitude lines appear as **straight and perpendicular** lines.
- **Pseudo-cylindrical** versions keep the central meridian and parallels straight, but other meridians become curved.

2..Planar (Azimuthal) Projections

- The sphere is projected onto a **flat plane** that touches the globe at one point (the center of the map).
- **Directional relationships** from the center are preserved.
- Some are **true perspective** projections, as if viewed from a point in space.

3.. Conic Projections

- The sphere is projected onto a **cone** placed over the globe.
- Parallels (latitude lines) become **circles**; meridians become **straight lines** from the center.
- Useful for mid-latitude regions (e.g., USA, Europe).
- Can be designed to **preserve distances** along certain lines.

Visual Variables for Spatial Data

Maps are used in many different ways: for example, to provide specific information about particular locations, to provide general information about spatial patterns, or to compare patterns in multiple maps. The mapping of spatial data properties to the visual variables must reflect this goal. The visual variables for spatial data are (see Figure 6.8):

- **Size** — size of individual symbols, width of lines, or size of symbols in areas
- **Shape** — shape of individual symbols or pattern symbols in lines and areas
- **Brightness** — brightness of symbols, lines, or areas
- **Color** — color of symbols, lines, or areas
- **Orientation** — orientation of individual symbols or patterns in lines and areas
- **Spacing (texture)** — spacing of patterns in symbols, lines, or areas

	Size	Shape	Brightness	Color	Orientation	Spacing	Perspective height	Arrangement
Point								
Linear								
Areal								

Visual variables for spatial data (Image based on [384].)

- **Perspective height** — perspective three-dimensional view of the phenomena with the data value mapped to the perspective height of points, lines, or areas
- **Arrangement** — arrangement of patterns within the individual symbols (for point phenomena), patterns of dots and dashes (for line phenomena), or regular versus random distribution of symbols (for area phenomena)

Eg:

The visualization also heavily depends on the extent of the areas for aggregation. Figure 6.10 shows the well-known London cholera example with different area aggregations, resulting in quite different maps.

Area Aggregation:



Different spatial aggregation yields different choropleth maps.

Fig:6.10

2.2. Visualization of Point Data

The first important class of spatial data is point data. Point data are discrete in nature, but they may describe a continuous phenomenon, for example, temperature measurements at specific locations. Depending on the nature of the data and the task, the designer must decide whether to display the data continuously, versus discrete and smooth, versus abrupt. Figure 6.11 show the different options. Discrete data are presumed to occur at distinct locations, while continuous data are defined at all locations. Smooth data refers to data that change in a gradual fashion, while abrupt data change suddenly.

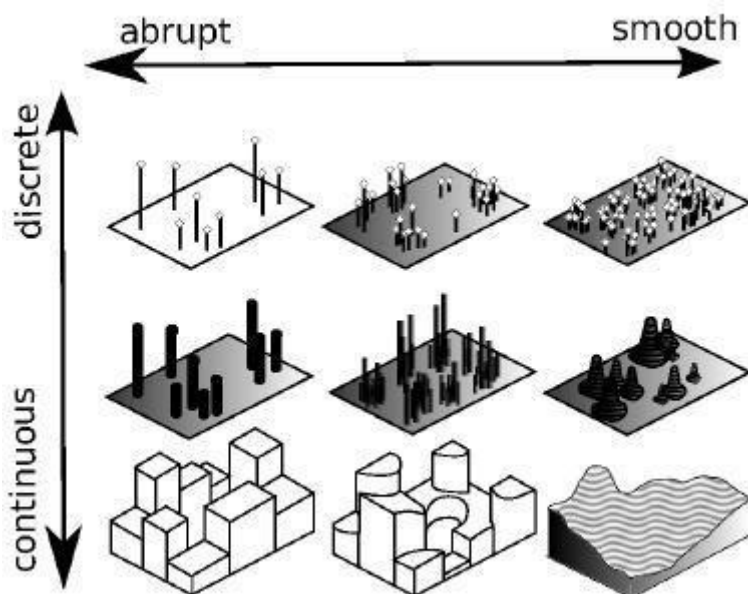


Figure 6.11. Discrete versus continuous and smooth versus abrupt

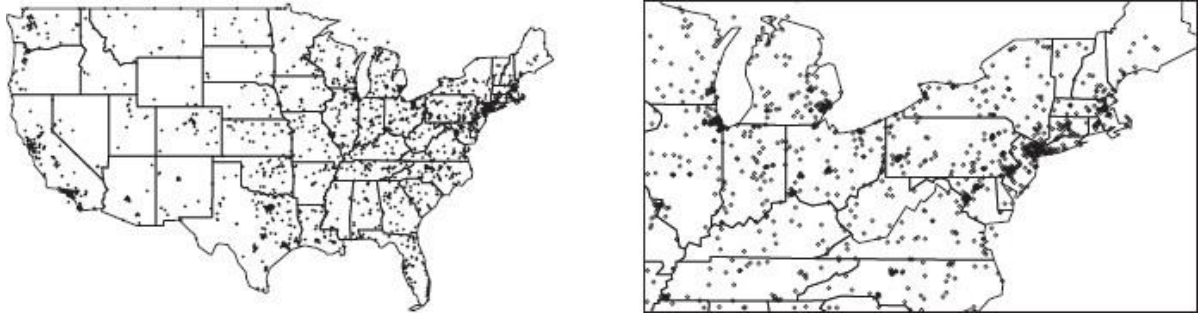
2.2.1 DOT MAPS :

Point phenomena can be visualized by placing a symbol or pixel at the location where that phenomenon occurs. This simple visualization is called a dot map. A quantitative parameter may be mapped to the size or the color of the symbol or pixel. Circles are the most widely used symbol in dot maps, but squares, bars, or any other symbol can be used as well. If the size of the symbol is used to represent a quantitative parameter, a specific question is how to scale the symbols. Calculating the correct size of the symbols does not necessarily mean that the symbols will be perceived correctly [129]. The perceived size of the symbols does not necessarily correspond to the actual size, due to problems in size perception). The perceived size of the symbols depends on their local neighborhood (e.g., the Ebbinghaus illusion , therefore no global formula for perceptual scaling is possible.

Maps are an elegant medium for communicating a wealth of information about the relationships of spatial point phenomena in a compact, convenient, and familiar format. However, when large data sets are drawn on a map, the problem of overlap or overplotting of data points arises in highly

populated areas, while low-population areas are virtually empty see Figure 6.12, since spatial data are highly non uniformly distributed in realworld data sets. Examples of such spatial data sets are credit card payments, telephone calls, health statistics, environmental records, crime data, and census

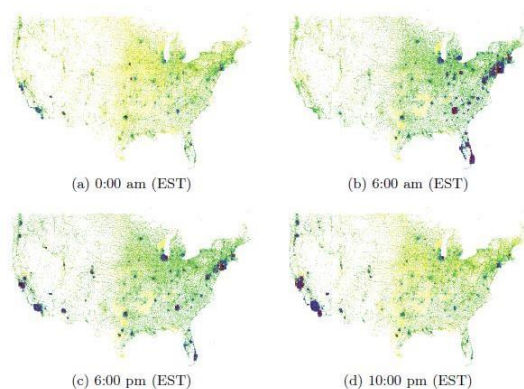
demographics. Note that the analysis may involve multiple parameters that may be shown on multiple maps. If all maps show the data in the same way, it may be possible to relate the parameters and detect local correlations,dependencies, and other interesting patterns.



6.12 USA dot map: every circle represents the spatial location of an event. Even in the zoomed-in version there is a large degree of overlap

2.2.1 PixelMaps:

The PixelMap approach visualizes point data without aggregation and prevents overlap in 2D maps by **repositioning pixels**. It recursively **partitions the dataset into four subsets** that correspond to **four subregions** of the display. If points do not fit in their assigned subregions, the **region boundaries are adjusted** while keeping the same subsets of points. A **quadtree-like data structure** is used to manage the recursive partitioning and point placement efficiently. This ensures that all points remain visible even in highly dense areas.



6.13 The figures display U.S. Telephone Call Volume at four different times during one day. The idea is to place the first data items at their correct position and position overlapping data points at nearby unoccupied positions.

A limitation of PixelMaps is that, in highly dense areas, the repositioning of points depends on the order of points in the dataset. An example is shown in Figure 6.13, which displays U.S. telephone call volume over four time steps within a 10-minute interval. The sequence helps users intuitively understand how call volume changes over time—such as increasing from east to west during morning wake-up hours and decreasing during commuting and lunchtime. While the results show expected spatial patterns, they also reveal unexpected ones, like dark spots over Kansas City, KS, and Omaha, NE, indicating call centers that operate nationwide at night.

2.3 Visualization of Line Data

To visualize spatial data that represents linear features, the data is shown as line segments connecting pairs of points defined by latitude and longitude. Different data values can be represented by changing the line's width, pattern, color, or by adding labels. The characteristics of the endpoints or intersections of the lines can also be shown visually by adjusting the size, shape, color, or labeling of those points. The lines do not have to be straight — they can be drawn as polylines or smooth curves to reduce clutter in the display. The best way to map these visual features depends on the specific application and the type of analysis required.

(a) Network Maps

Network maps are commonly used in many applications to show how different components are connected and to help users understand the overall structure and behavior of a network. Some visualization approaches focus only on displaying the connectivity. For example, Eick and Wills [107] used aggregation, hierarchical information, node positioning, and linked displays to explore large networks that may not have a natural layout, and they applied color and shape to represent node properties while using color and line width to show link information. Researchers at NCSA [305] introduced 3D graphics to animate Internet traffic moving through the network backbone. Becker, Eick, and Wilks [28] developed a system called SeeNet, which encourages user interaction so that patterns of interest can be explored more easily. SeeNet uses geographic displays along with a link matrix that gives equal visual importance to all network connections. Another major system is AT&T's SWIFT-3D [224, 253, 254], which combines multiple visualization techniques—including statistical graphics, pixel-based overviews, and interactive 3D network maps—together with drag-and-drop tools for flexible querying of large network data.

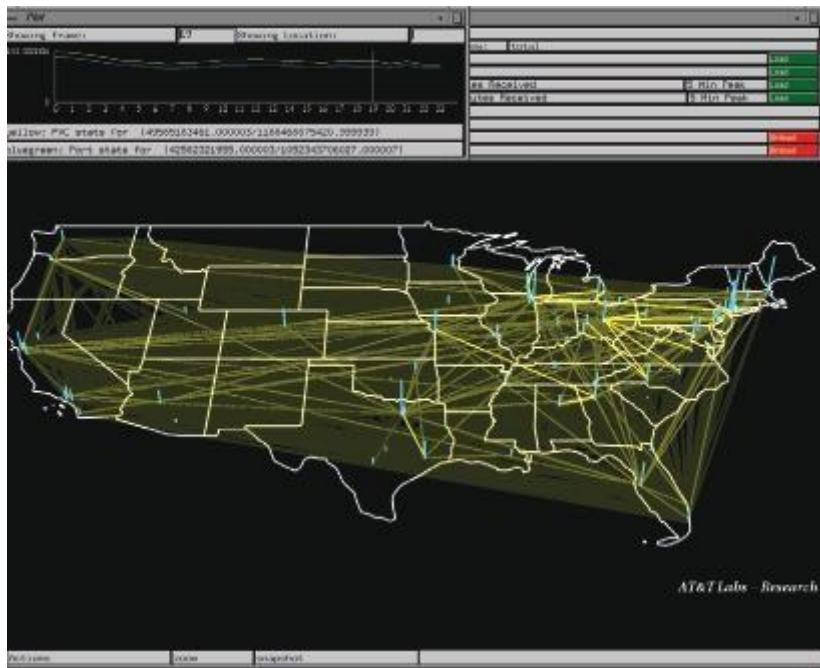


Figure 6.14. Swift-3D

Fig: Swift -3D.

(b) Flow Maps and Edge Bundling

Several approaches have been developed to reduce the overlap problem in traditional network maps by using **curved lines** instead of straight lines (see Figure 6.15). While cartographers originally created such curved visualizations manually, more recent methods generate them **algorithmically**. Two well-known examples of these automated techniques are **Stanford flow maps** [318] and **Danny Holten's edge bundling**.



6.15 Fig: Arc Map

The flow map technique is based on **graph layout algorithms** that aim to minimize edge crossings and distortions in node positions while keeping their relative locations intact. The method uses **hierarchical clustering** of nodes and the flows between them to determine how flows can be merged and rerouted effectively. Compared to other computer-generated flow maps (Figure 6.16(a)), the **Stanford system** produces a much clearer visualization with significantly reduced clutter (Figure)

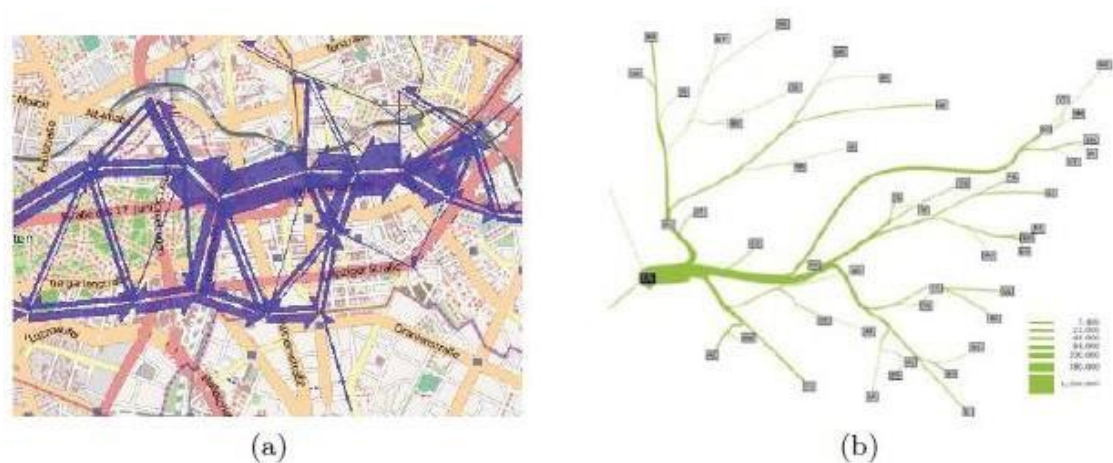


Fig: 6.16 Flow maps: (a) flows of tourists in Berlin; (b) produced by the Stanford system showing the migration from California

Fig: Flow maps : Flow of tourists in Berlin

Edge bundling is another technique designed to reduce clutter in line drawings. When a **hierarchy** is defined on the nodes, edges can be bundled according to that hierarchy using **B-splines** to connect two nodes. Edges passing through the root of the hierarchy are bent the most, while edges within the same subhierarchy are bent only slightly. This **hierarchical bundling** greatly reduces visual clutter and is a versatile method that can be applied to many visualization types, including traditional maps, tree diagrams, circular trees, and treemaps. For example, Figure 6.17 shows edge bundling applied to visualize **IP traffic data**.

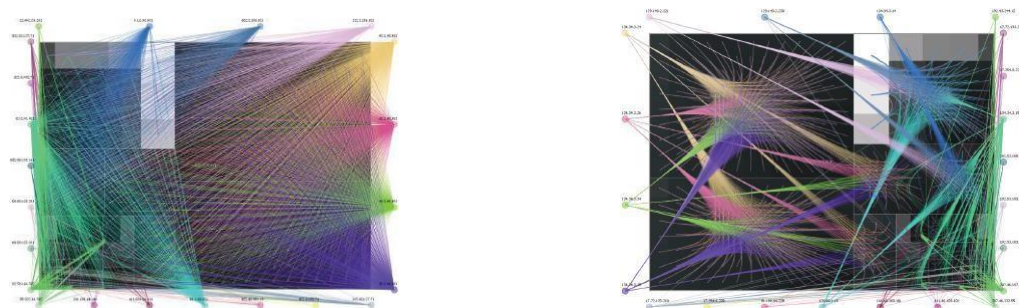


Fig:The visualizations show IP flow traffic from external nodes on the outside to internal nodes, visualized as treemaps on the inside. The edge bundling visualization(right side) significantly reduces the visual clutter compared to the straight line visualization (left side).

2.4 Visualization of Area Data:

Thematic maps are the main method for visualizing area-based phenomena and come in several variants. The most common type is the **choropleth map** (Greek: choro = area, pleth = value), where attribute values are shown using colors or shades for different regions (Figure 6.18(a)). Choropleth maps assume the attribute is evenly distributed within each region, but when this is not the case, **dasymetric maps** are used, which create areas based on the attribute itself rather than pre-defined regions (Figure 6.19). Another important type is the **isarithmetic map** (Figure 6.18(b)), which depicts contours of continuous phenomena, such as temperatures or elevation. If the contours are derived from measurements at specific points, the maps are called **isometric maps**; if from regional averages using centroids, they are called **isopleth maps**. Creating isarithmic maps usually requires **interpolation** techniques, such as triangulation or inverse distance mapping, to generate smooth contours. A more complex but less common approach is the **cartogram**, where the sizes of regions are scaled to reflect a statistical variable, resulting in distorted map geometry. Cartograms can be **continuous**, retaining the topology of the regions, or **noncontinuous**, scaling each area independently to approximate rectangles or circles.

Choropleth Maps

Choropleth maps usually present the area phenomena as **shaded polygons** that are closed contours of sets of points, where the first and the last points are the same. Examples of closed contours are **states, counties, and parks**. Choropleth maps are used to emphasize the **spatial distribution** of one or more geographic attributes. In generating choropleth maps, the **normalization of the data**, as well as **color or grayscale mapping**, are important design decisions. In Fig 6.18(a), a choropleth map showing the **2008 presidential election results** is shown.

A problem of choropleth maps is that the most interesting values are often concentrated in **densely populated areas** with small and barely visible polygons, while less interesting values are spread out over **sparsely populated areas** with large and visually dominating polygons. Choropleth maps, therefore, tend to highlight patterns in **large areas**, which may, however, be of lower importance. In the **U.S. Census Demographics**, for example, such maps tend to highlight patterns in areas where few people live, e.g., the large states in the USA.

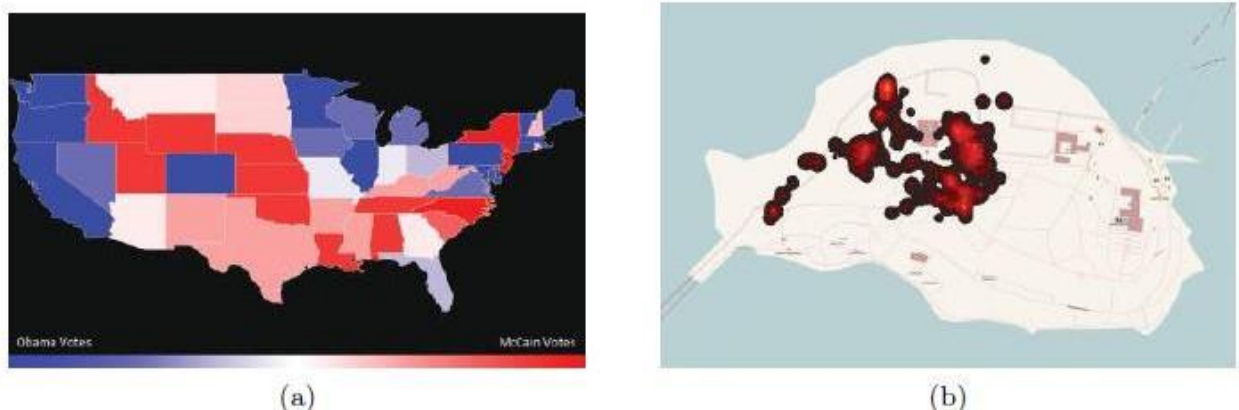


Fig 6.18 Thematic maps: (a) A choropleth map showing U.S. election results of the 2008 Obama versus McCain presidential election. (b) An isarithmic map showing the number of pictures taken on Mainau Island, using a heatmap, where the colors range from black to red to yellow, with yellow representing the most photographs.

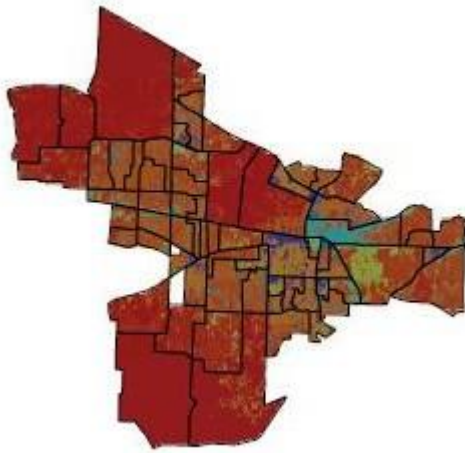


Fig 6.19 A dasymetric map showing the population distribution in Beaverton Creek, Oregon, USA.

Cartograms:

Cartograms are generalizations of ordinary thematic maps that avoid the problems of choropleth maps by **distorting the geography** according to the displayed statistical value. Cartograms are a specific type of map transformation, where the regions are **resized according to a geographically related input variable**. Example applications include **population demographics, election results, and epidemiology**.

Several categories of cartogram problems exist. As shown in Figure , **noncontinuous cartograms** can exactly satisfy area and shape constraints, but don't preserve the input map's topology. Because the scaled polygons are drawn inside the original regions, the **loss of topology** doesn't cause perceptual problems. More critical is that the polygon's **original size restricts its final size**, so small polygons cannot be made arbitrarily large without scaling the entire map, which can make important areas difficult to see and reduce screen usage efficiency.

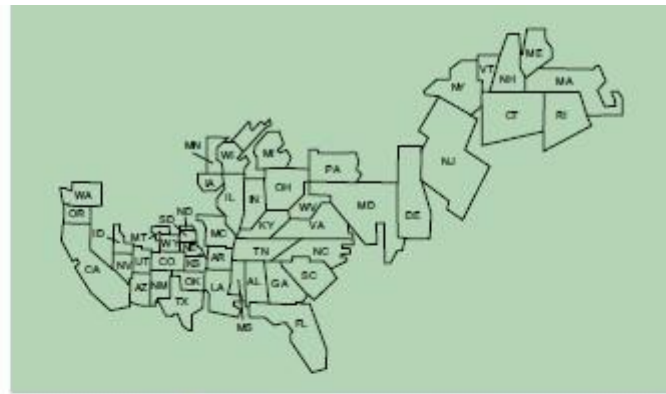
Noncontiguous cartograms, shown in Figure 6.20(b), scale all polygons to their **target sizes**, perfectly satisfying the area objectives. Shapes can be slightly relaxed so polygons touch without overlapping, and the map's topology is highly relaxed because polygons **don't retain adjacency relationships**. Noncontiguous cartograms provide perfect area adjustment with good shape preservation, but lose the map's **global shape and topology**, making it harder to perceive the visualization as a map.

Circular cartograms, shown in Figure 6.20(c), completely ignore the input polygon's shape, representing each as a **circle** in the output. In many cases, area and topology constraints are relaxed, so circular cartograms share some of the same problems as noncontiguous cartograms.

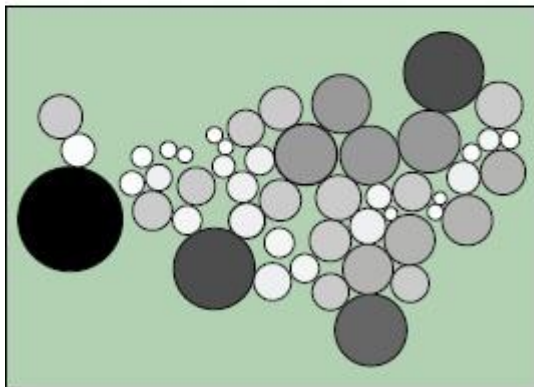
Continuous cartograms, shown in Figure 6.20(d), retain a map's **topology perfectly**, but relax the given area and shape constraints. In general, cartograms cannot fully satisfy both shape and area objectives, so generating them involves a **complex optimization problem** to balance shape and area preservation. Although continuous cartograms are difficult to generate, the resulting **polygonal meshes** resemble the original map more closely than other computer-generated cartogram variants. Therefore, the rest of this section focuses on **continuous cartograms**.



(a) Noncontinuous cartogram.



(b) Noncontiguous cartogram.



(c) Circular cartogram.



(d) Continuous cartogram.

6.20 Different types of cartograms.

2.5 Other Issues in Geospatial Data Visualization

A prerequisite to generating useful visualizations of spatial phenomena are a number of well-known techniques from **cartography**, including **map generalization** and **map labeling**.

Map generalization is the process of selecting and abstracting information on a map. Generalization is used when a **small-scale map** is derived from a **large-scale map** containing detailed information. The goal is to adapt the objects on the map so that the objects of interest may be easily perceived in the resulting visualization. Map generalizations are **application- and task-dependent**; good generalizations emphasize the elements most important for the task while still representing the geography accurately and recognizably. It is important that the **visibility and recognizability** of objects outweigh the loss of detail due to generalization.

Examples of typical map generalizations include:

- **Simplify points** — remove or combine points that are not relevant or not separately visible on the small-scale map.
- **Simplify lines** — remove small fluctuations and bends, or collapse dual lines to centerlines. Lines that risk touching in the scaled map may be removed.
- **Simplify polygons** — remove small fluctuations and bends while preserving the essential shape. This may include simplifying building footprint boundaries while retaining size and shape, or combining disjoint but adjacent polygons into a new area based on a tolerance distance.

Map labeling involves placing textual or figurative labels close to points, lines, and polygons. While it may seem straightforward, it is a **difficult problem**. Many label placement algorithms exist, differing in **effectiveness** (quality of results) and **efficiency** (speed of calculation). Most algorithms use **heuristics**, typically starting with rule-based placement followed by an **optimization algorithm**, such as local search, greedy algorithms, simulated annealing, random placement, or genetic algorithms. Examples of labeling software include **Label-EZ** by MapText, Inc., and **Maplex** by ESRI.

Other important issues relate to the design of **effective Geographic Information Systems (GIS)**. A GIS allows users to create interactive queries for dynamic search and exploration, compare and edit spatial data across different map layers, and present the results of these operations. **Geospatial data visualization** is a part of GIS, and advances in visualization and other GIS functions can mutually enhance each other, making the system more powerful and useful.

In recent years, with the advance of **fast-growing web technology and APIs** (e.g., Flex, AJAX, and the Google MAP API), as well as the **public availability of digitized geographic data** and various economic, social, and environmental measures and indicators via the Internet, the GIS community has developed a large number of **interactive, high-performance tools** for spatial data visualization. These tools significantly increase awareness and foster a better understanding of **public issues** among large populations. As a result, the **visualization of geospatial data** has become exciting and engaging to many people.